

UNIT-I

STACK APPLICATIONS AND TREES

Data Structure is a way to store and organize data so that it can be used efficiently.

OR

Data Structure can be defined as the group of data elements which provides an efficient way of storing and organizing data in the computer so that it can be used efficiently.

Advantages of Data structures:

Efficiency

If the choice of a data structure for implementing a particular ADT is proper, it makes the program very efficient in terms of time and space

Reusability

The data structure provides reusability means that multiple client programs can use the data structure.

Abstraction

The data structure specified by an ADT also provides the level of abstraction. The client cannot see the internal working of the data structure, so it does not have to worry about the implementation part. The client can only see the interface

There are two types of data structures:

1. Primitive data structure
2. Non-primitive data structure

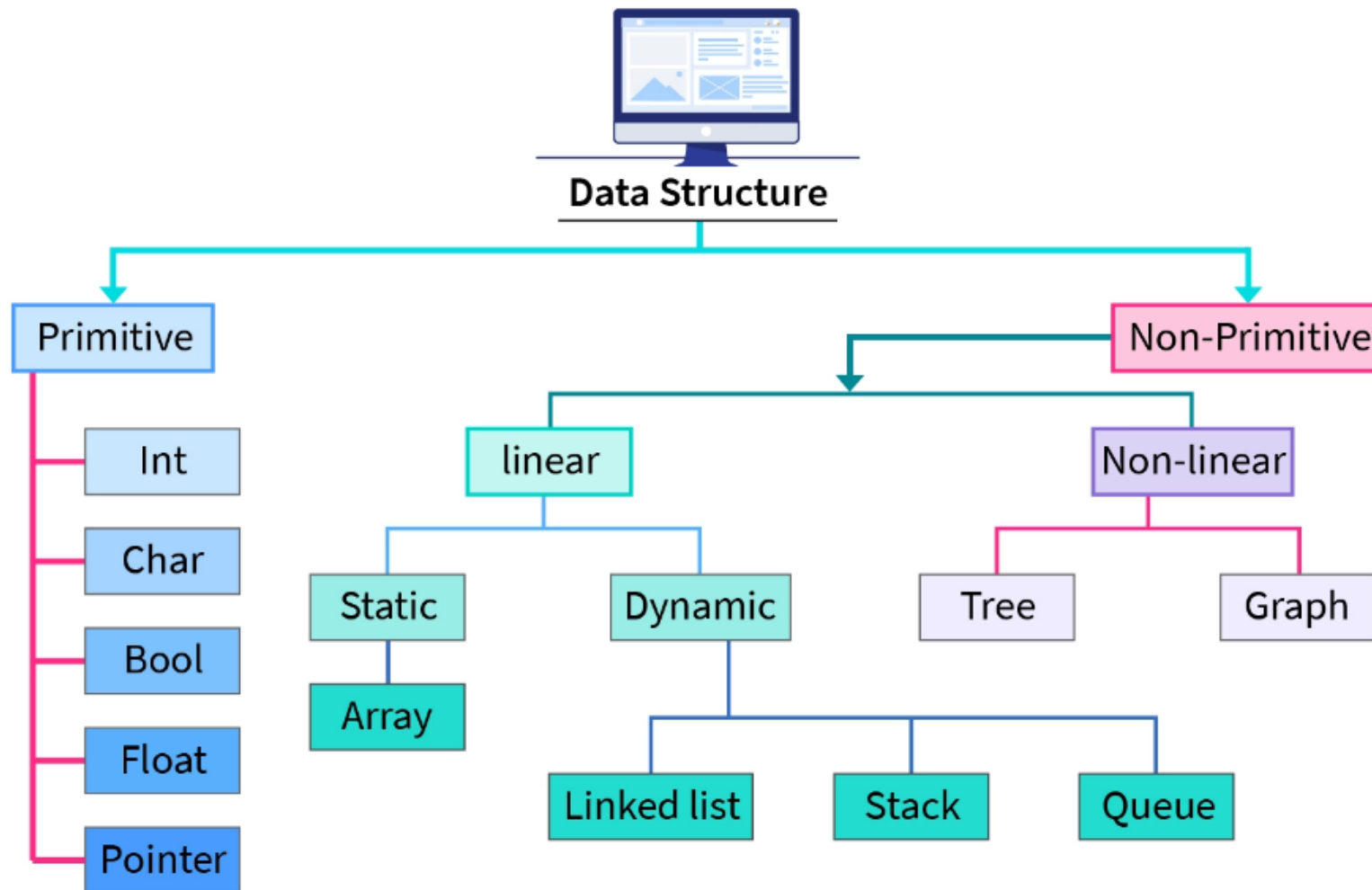
Primitive Data structure: The primitive data structures are primitive data types. The **int, char, float, double, and pointer** are the primitive data structures that can hold a **single value**.

Non-Primitive Data structure: The non-primitive data structure is divided into two types:

1. Linear data structure
2. Non-linear data structure

Linear Data Structure: The arrangement of data in a sequential manner is known as a linear data structure. The data structures used for this purpose are **Arrays, Linked list, Stacks, and Queues**. In these data structures, one element is connected to only one another element in a linear form.

Non-Linear Data Structure: When one element is connected to the 'n' number of elements known as a non-linear data structure. The best example is **trees and graphs**. In this case, the elements are arranged in a random manner.

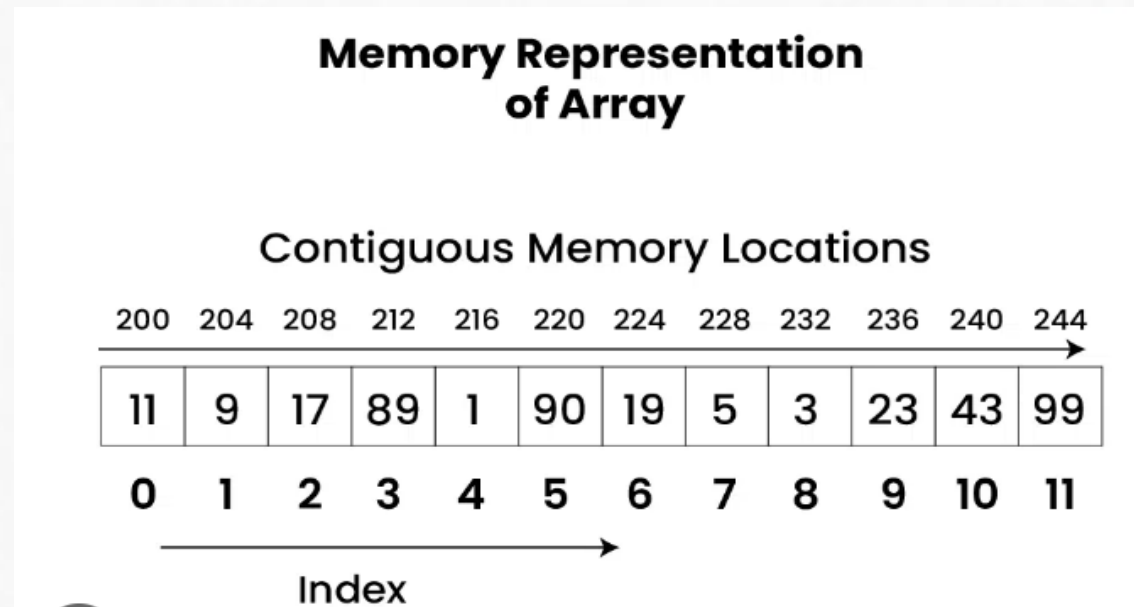


Static Representation of Data structures:

Static data structure:

- In Static data structure the size of the structure is fixed. It is a type of data structure where the size is allocated at the compile time.
- Therefore, the maximum size is fixed. The content of the data structure can be modified but without changing the memory space allocated to it.

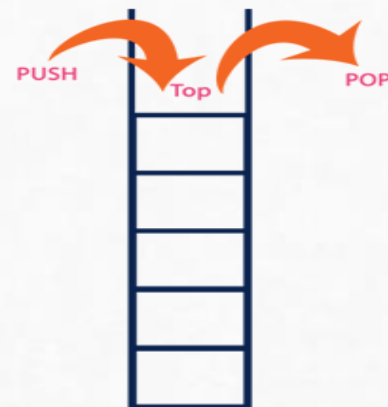
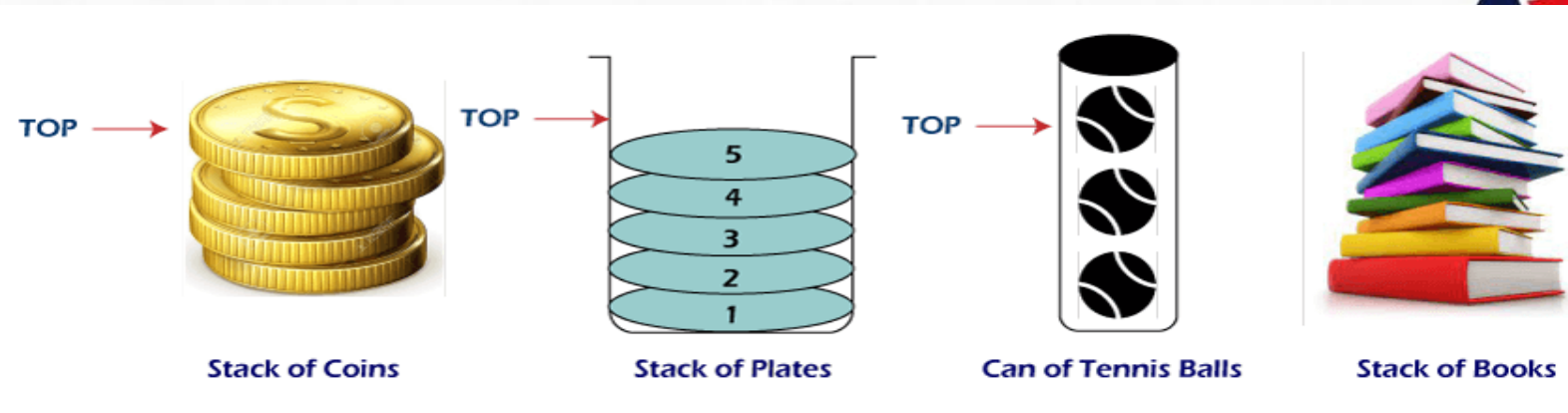
Example: Array.



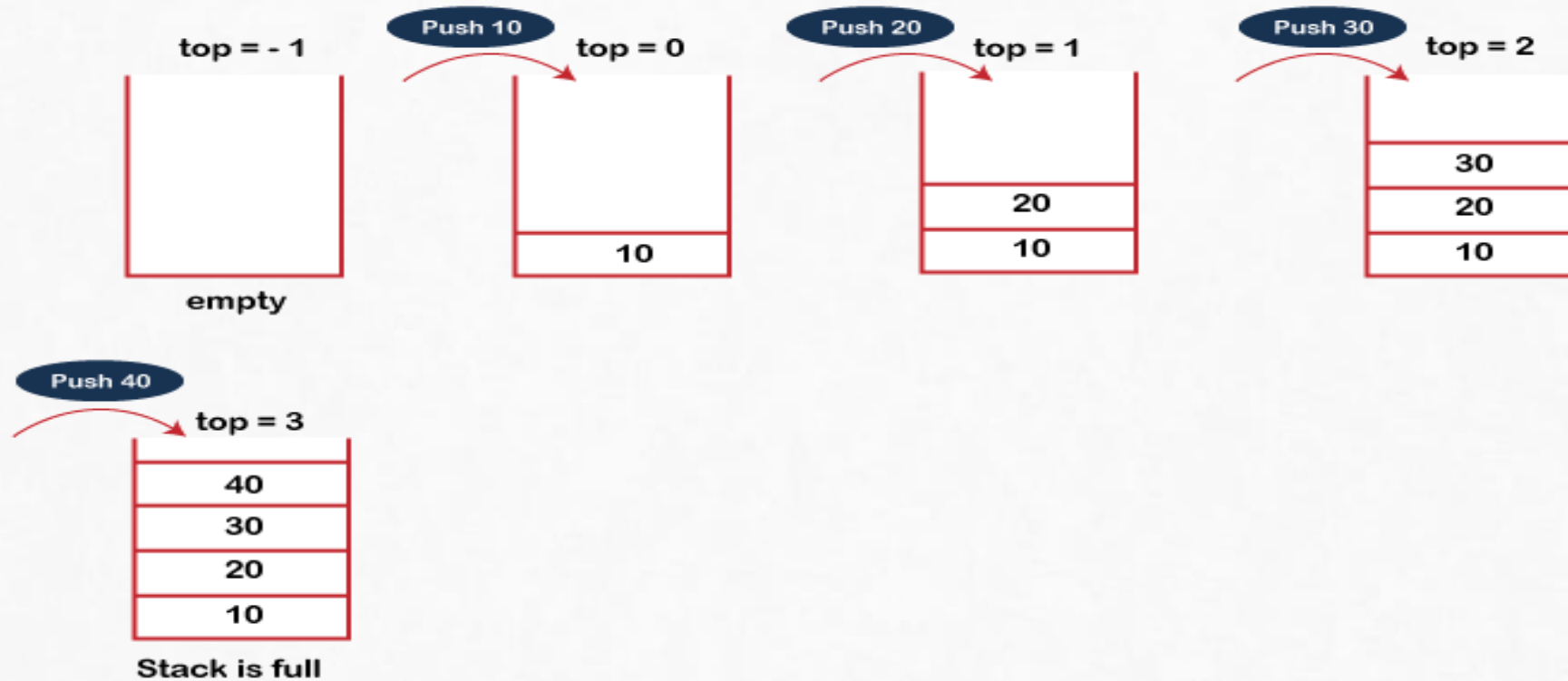
Stack Definition:

“A stack is a linear data structure. In which, insertions and deletions are takes place at one end, called the top. Stack maintains a variable called top, which keeps track of the top most elements in the stack”.

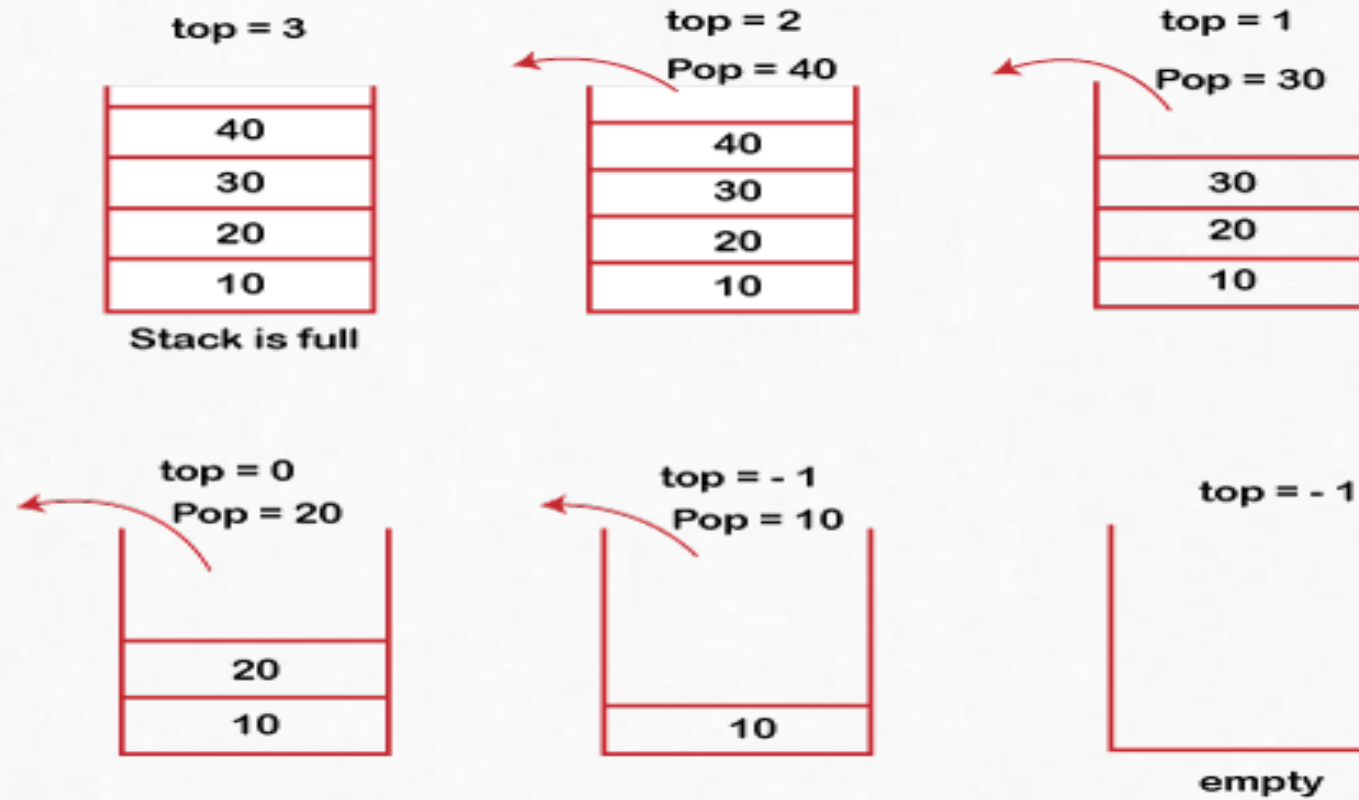
Stacks are sometimes called as **Last-In-First-Out (LIFO)** lists i.e. the element which is inserted first in the stack, will be deleted last from the stack”



PUSH operation



POP operation:



Top and its value

Top position	Status of stack
-1	Empty
0	Only one element in the stack
N-1	Stack is full
N	Overflow

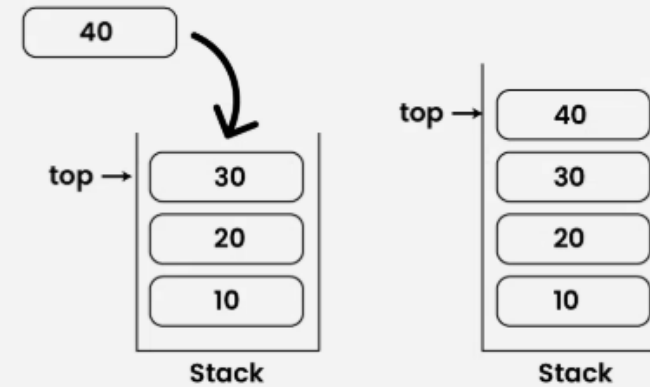
Stack implementation using Arrays:

Push operation

Algorithm:

- 1.begin
- 2.if $\text{top} = n-1$ then stack full
3. $\text{top} = \text{top} + 1$
4. $\text{stack}(\text{top}) := \text{item};$
- 5.end

Push Operation in Stack

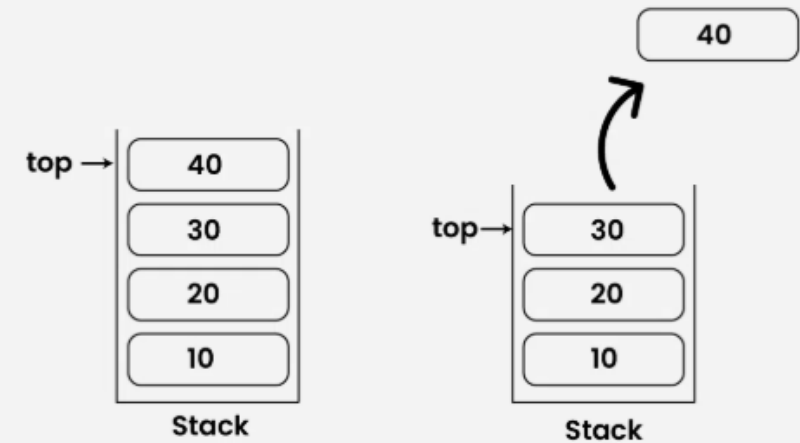


Pop operation

Algorithm:

1. begin
2. if $\text{top} = -1$ then stack empty;
3. $\text{item} := \text{stack}(\text{top});$
4. $\text{top} = \text{top} - 1;$
5. end;

Pop Operation in Stack

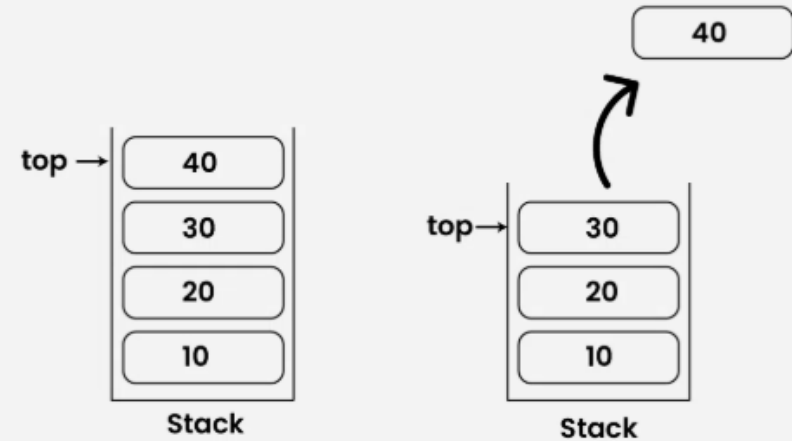


Algorithm:

PEEK (STACK, TOP)

1. Begin
2. if $\text{top} = -1$ then stack empty
3. $\text{item} = \text{stack}[\text{top}]$
4. return item
5. End

Pop Operation in Stack



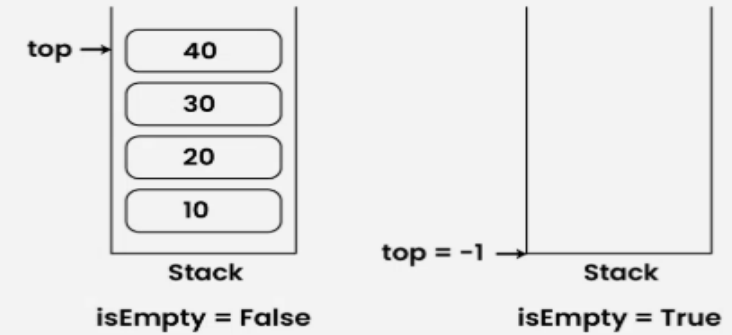
isEmpty Operation

Returns true if the stack is empty, else false.

Algorithm for isEmpty Operation:

1. Check for the value of **top** in stack.
2. If (**top == -1**), then the stack is **empty** so return **true**.
3. Otherwise, the stack is not empty so return **false**.

isEmpty Operation in Stack



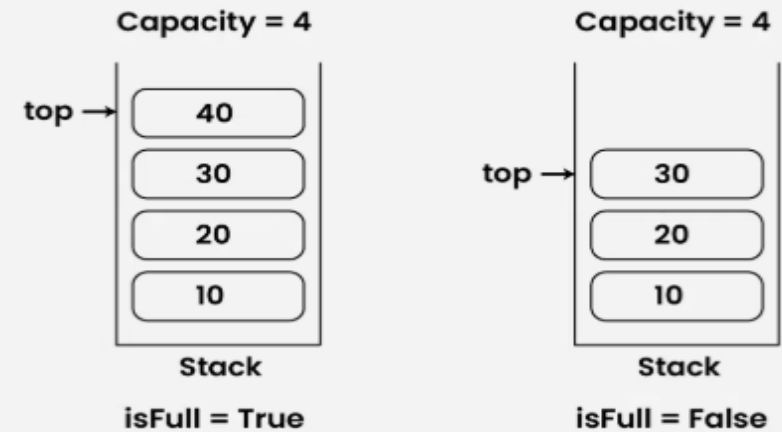
isFull Operation

Returns true if the stack is full, else false.

Algorithm for isFull Operation:

1. Check for the value of **top** in stack.
2. If (**top == capacity-1**), then the stack is **full** so return **true**.
3. Otherwise, the stack is not full so return **false**.

isFull Operation in Stack



Applications of stacks:

- **Expression Evaluations & Conversions:** A stack is a very effective data structure for evaluating arithmetic expressions in programming languages. An arithmetic expression consists of operands and operators.
- **Backtracking(Recursion):** Backtracking is another application of Stack. It is a recursive algorithm that is used for solving the optimization problem.
- **Parsing (Delimiter Checking) :** The common application of Stack is delimiter checking, i.e., parsing that involves analyzing a source program syntactically. It is also called **parenthesis checking**.
- **Function calls:** Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing.
- **Editors:** Undo and Redo functions in any text editor.
- **Tree Traversals:** Stacks are useful for Depth First Search Traversal method.
- **Browsers:** Stacks are useful for function calls, storing the activation records. The history of a web browser is stored in the form of a stack.

Expression Types:

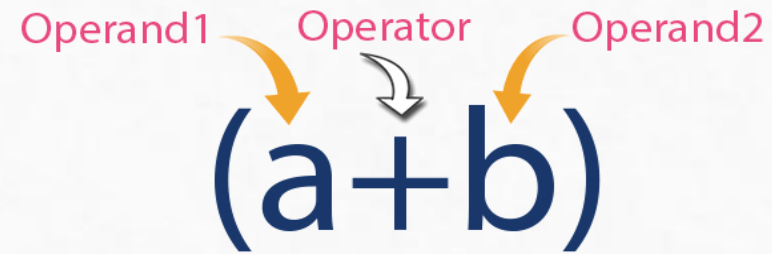
Based on the operator position, expressions are divided into THREE types. They are as follows...

1.Infix Expression

2.Postfix Expression

3.Prefix Expression

Infix Expression:



Postfix Expression:



Prefix Expression:



Order of Operations (Precedence) from **left to right**

Operator		Priority
Parenthesis	()	Highest
Exponentiation	^	
Unary: -, NOT ~		
Multiplication/division:	* / \	
Addition/subtraction:	+ -	
Relational:	<, <=, >, >=, ==, ~=	
AND:	&&	
OR:		
Assignment:	=	Lowest

^

Exponentiation right to left

Representation of infix to Postfix Expression (Reverse Polish notation) using stack:

1. Read all the symbols one by one from **left to right** in the given Infix Expression.
2. If the reading symbol is **operand**, then directly print it to the result (Output).
3. If the reading symbol is left parenthesis '(', then **Push it on to the Stack**.
4. If the reading symbol is right parenthesis ')', **pop the stack and print** the operators until left parenthesis is found.
5. if the reading symbol is the operator and the Stack is empty or contains the '(', ')' symbol, push the operator into the Stack.
6. If incoming symbol has **higher precedence than the top of the stack**, push it on the stack
7. If incoming symbol has **lower precedence than the top of the stack**, pop and print the top. Then test the incoming operator against the new top of the stack.
8. If incoming operator has **equal precedence with the top of the stack**, use associatively rule
 - ✓ If associatively **L to R** then pop and prints the top of the stack and then push the incoming operator.
 - ✓ If associatively **R to L** then push the incoming operator
9. At the end of the expression, pop and print all operators of stack.

Evaluation of **Postfix** Expression **using Stack**:

Algorithm:

- **Step-1:** Create an empty STACK and start scanning the Postfix expression from Left to Right.
- **Step-2:** if the element is an Operand, PUSH it into the STACK.
- **Step-3:** if the element is an **Operator**, POP twice and get top most two elements A and B; Calculate **B Operator A** and PUSH the result back into the STACK.
- **Step-4:** When the expression is ended, the value in the STACK is the final answer.

Example:01

$$\begin{aligned} \text{i) } 4 + 6 * 2 &=> 4 + (6 * 2) \\ &= (4 * 6 + 2) \\ &= 4 * 6 + 2 \end{aligned}$$

$$\begin{aligned} \text{iii) } 2 * 6 / 2 - 3 + 7 &=> (2 * 6) / 2 - 3 + 7 \\ &= (2 * 6 * 2 /) - 3 + 7 \\ &= (2 * 6 * 2 / 3 -) + 7 \\ &= (2 * 6 * 2 / 3 - 7 +) \\ &= 2 * 6 * 2 / 3 - 7 + \end{aligned}$$

$$\begin{aligned} \text{ii) } 2 + 3 * 4 - 5 &=> 2 + (3 * 4) - 5 \\ &= (2 * 3 * 4 +) - 5 \\ &= (2 * 3 * 4 * 5 + -) \\ &= 2 * 3 * 4 * 5 - \end{aligned}$$

Towers of Hanoi

- Tower of Hanoi, is a mathematical puzzle which consists of three towers (pegs) and more than one rings is as depicted.
- These rings are of different sizes and stacked upon in an ascending order, i.e. the smaller one sits over the larger one.
- There are other variations of the puzzle where the number of disks increase, but the tower count remains the same

Rules:

The mission is to move all the disks to some another tower without violating the sequence of arrangement. A few rules to be followed for Tower of Hanoi are

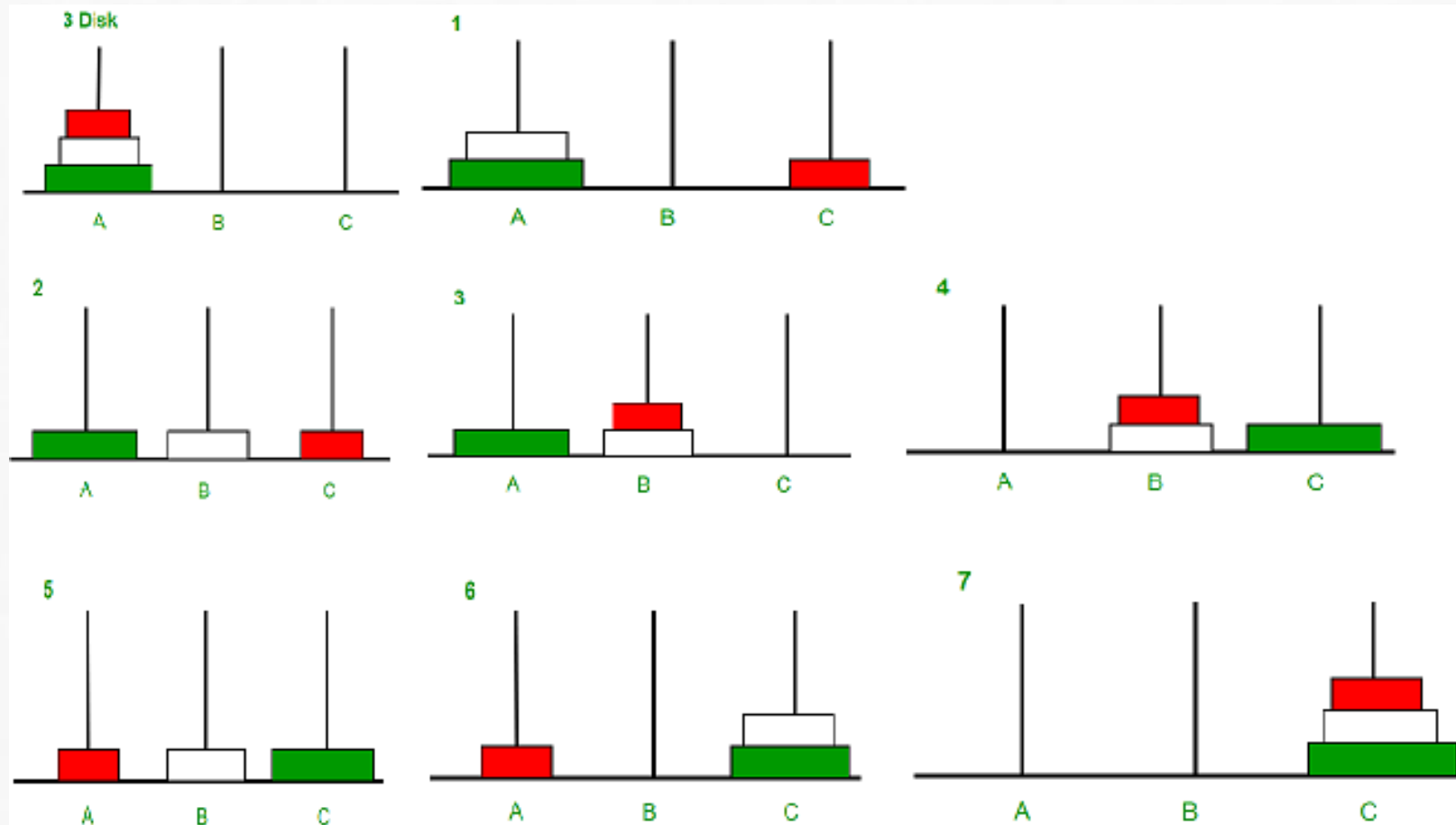
- ✓ **Only one** disk can be moved at a time.
- ✓ Each move consists of **taking the upper disk from one of the stacks** and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack.
- ✓ **No disk may be placed on top of a smaller disk.**

Example: Input: $N = 3$,

Step 1 – Move $N-1$ disks from A to B, using C.

Step 2 – Move Last (Nth) disk from A to C.

Step 3 – Move $N-1$ disks from B to C, using A



A recursive algorithm

START

Procedure Hanoi(disk, source, dest, aux)

IF disk == 1, THEN

 move disk from source to dest

ELSE

 Hanoi(disk - 1, source, aux, dest) // Step 1

 move disk from source to dest // Step 2

 Hanoi(disk - 1, aux, dest, source) // Step 3

END IF

END Procedure

STOP

Tower of Hanoi puzzle with n disks can be solved in minimum $2^n - 1$ steps. This presentation shows that a puzzle with 3 disks has taken $2^3 - 1 = 7$ steps

Example 1: $2 * ((4/2) + 5)$

The above expression has two opening and two closing brackets which means that the above expression is a balanced parenthesis.

Example 2: $2 * ((4/2) + 5$

The above expression has two opening brackets and one closing bracket, which means that both opening and closing brackets are not equal; therefore, the above expression is unbalanced

```
// Function to check if the expression is balanced
int isBalanced(char* text)
{
    int i;
    for (i = 0; i < strlen(text); i++)
    {
        if (text[i] == '(' || text[i] == '[' || text[i] == '{')
        {
            push(text[i]);
        }
        else if (text[i] == ')' || text[i] == ']' || text[i] == '}')
        {
            if (top == -1) {
                return 0; // If no opening bracket is present
            }
            else if (!is_matching_pair(pop(), text[i])) {
                return 0; // If closing bracket doesn't match the
                           last opening bracket
            }
        }
    }
    if (top == -1) {
        return 1; // If the stack is empty, the expression is balanced
    } else {
        return 0; // If the stack is not empty, the expression is not balanced
    }
}
```

```
// Main function
int main() {
    char text[MAX_SIZE];
    printf("Input an expression in parentheses: ");
    scanf("%s", text);

    // Check if the expression is balanced or not
    if (isBalanced(text)) {
        printf("The expression is balanced.\n");
    } else {
        printf("The expression is not balanced.\n");
    }
    return 0;
}
```

Input an expression in parentheses: ((({})))
The expression is balanced.

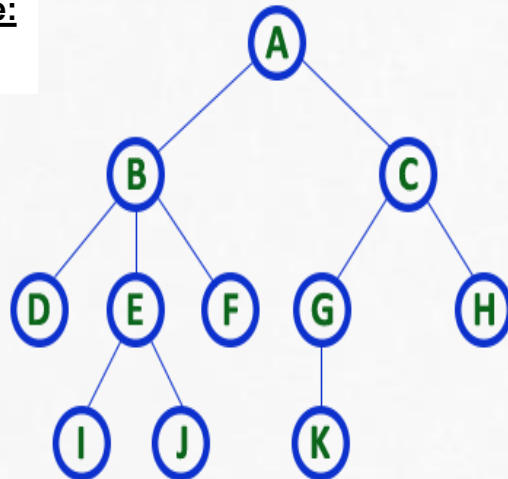
TREES

Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.

A tree data structure can also be defined as follows...

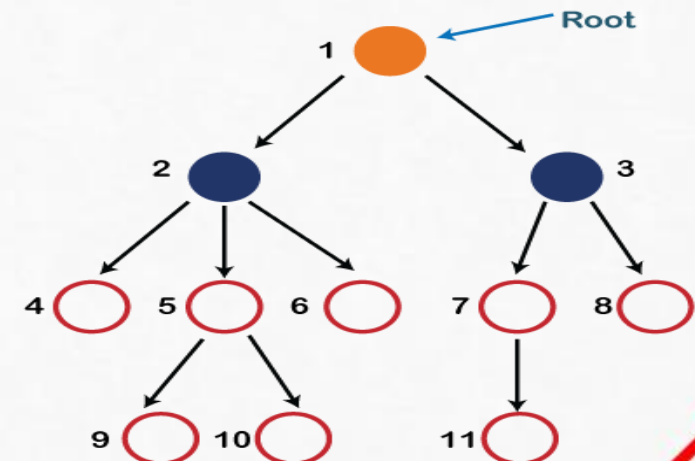
A **tree** is a finite set of **nodes** together with a finite set of directed **edges** (links/branches) that define parent-child (*HIERARCHICAL*) relationships

Example:



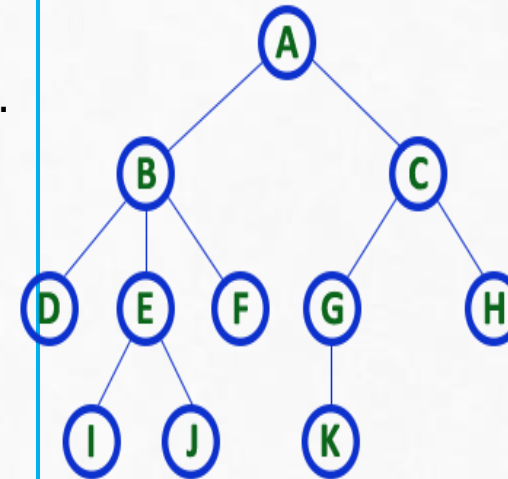
TREE with 11 nodes and 10 edges

- In any tree with ' N ' nodes there will be maximum of ' $N-1$ ' edges
- In a tree every individual element is called as '**NODE**'



A tree satisfies the following properties:

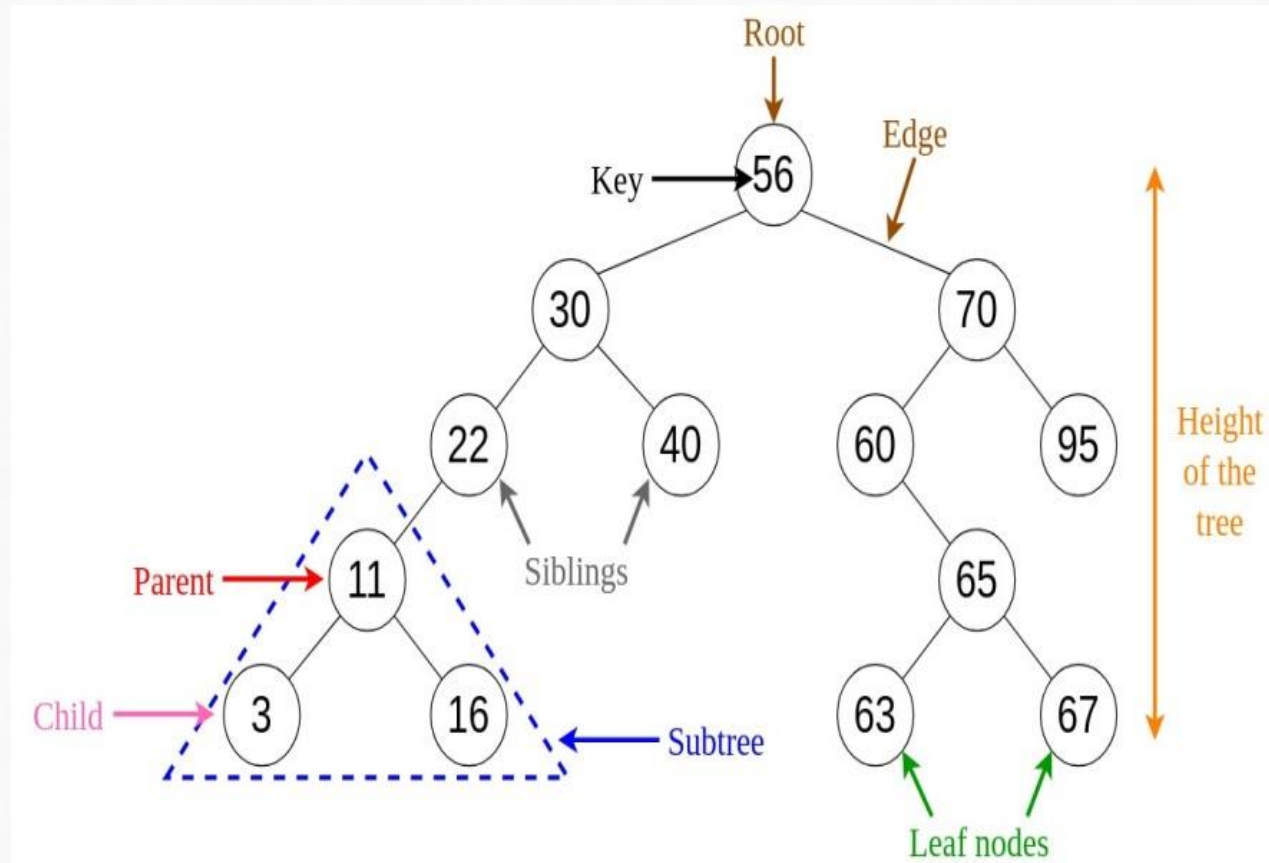
- It has one designated node, called the root that has no parent.
- Every node, except the root, has exactly one parent.
- A node may have zero or more children.
- There is a unique directed path from the root to each node.



TREE with 11 nodes and 10 edges

- In any tree with ' N ' nodes there will be maximum of ' $N-1$ ' edges
- In a tree every individual element is called as '**NODE**'

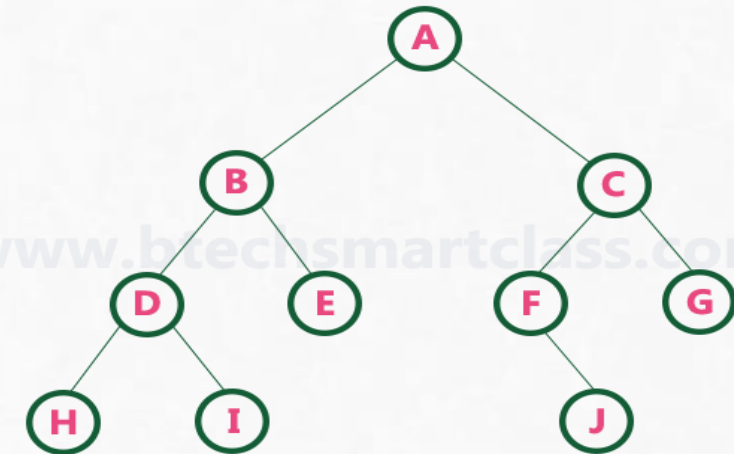
1. Root:
2. Parent:
3. Edge:
4. Child:
5. Degree:
6. Internal Nodes:
7. Leaf:
8. Siblings:
9. Level:
10. Height:
11. Depth:
12. Path:
13. Sub Tree:



A tree in which every node can have a maximum of two children is called Binary Tree

Binary Tree Properties:

- ✓ A binary tree with n elements, $n > 0$, has exactly $N-1$ edges.
- ✓ A binary tree of height H , $H \geq 0$, has at least h and at most $2^{h+1}-1$ elements or nodes in it.
- ✓ The height of a binary tree that contains N elements, $N \geq 0$, is at least $(\log_2(n+1))$ and at most n .

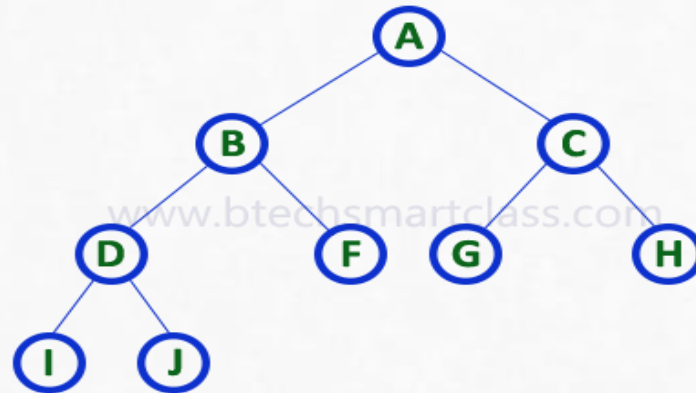


Different types of binary trees

1. Full Binary (Strictly Binary) Tree:

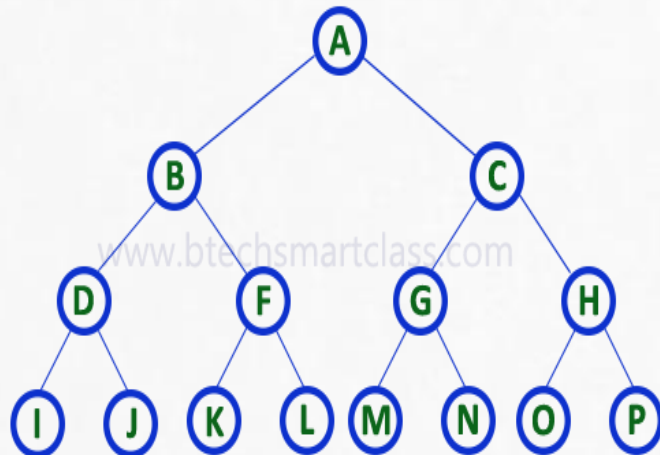
A binary tree in which every node has either two or zero number of children is called Strictly Binary Tree

Strictly binary tree is also called as **Full Binary Tree** or **Proper Binary Tree** or **2-Tree**



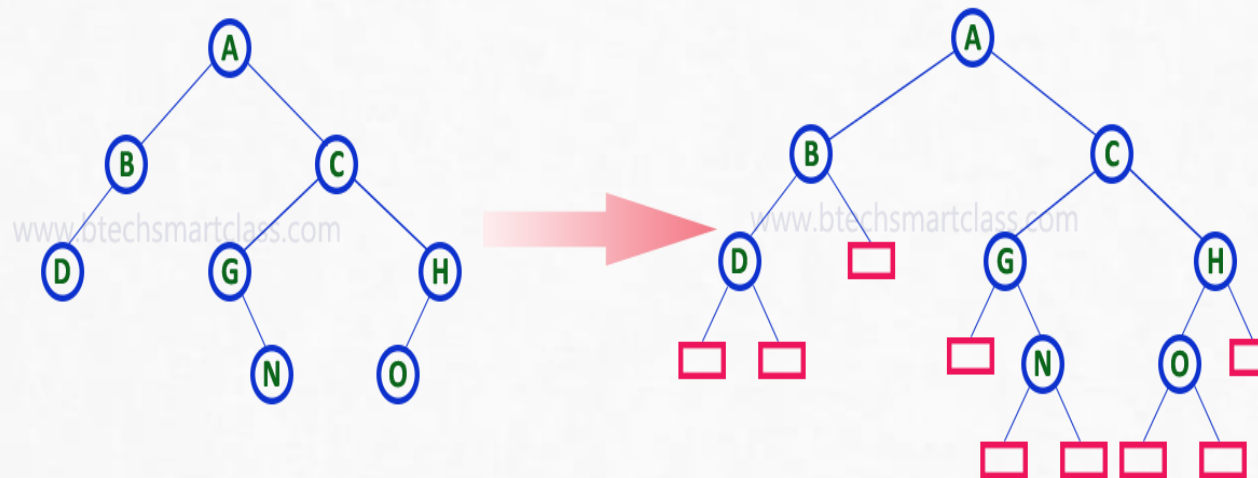
2. Complete Binary Tree:

A complete binary tree is a binary tree in which every level is completely filled except possibly the last level. In the unfilled level, the nodes are attached starting from the left-most position



3. Extended Binary Tree:

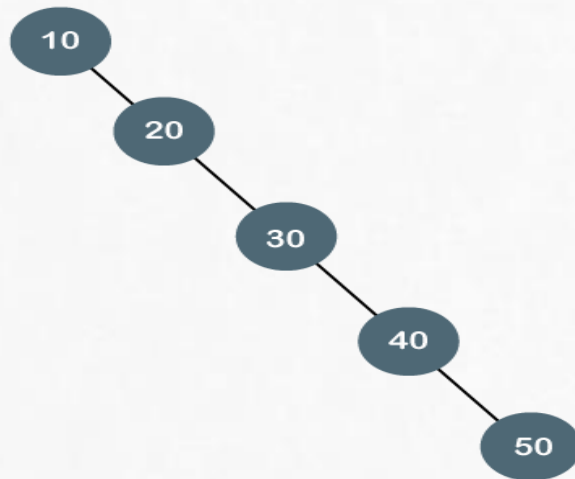
The full binary tree obtained by adding dummy nodes to a binary tree is called as Extended Binary Tree



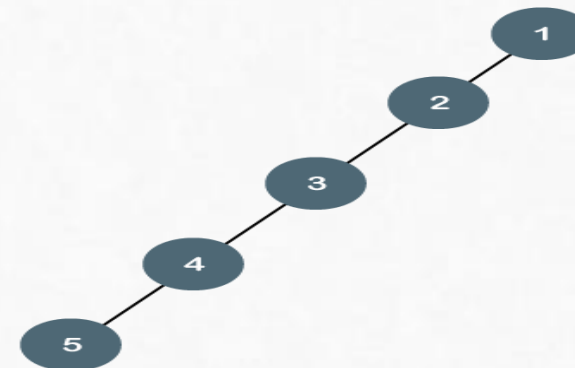
4. Degenerate Binary Tree:

The degenerate binary tree is a tree in which all the internal nodes have only one children.

Let's understand the Degenerate binary tree through examples.



right-skewed



left-skewed

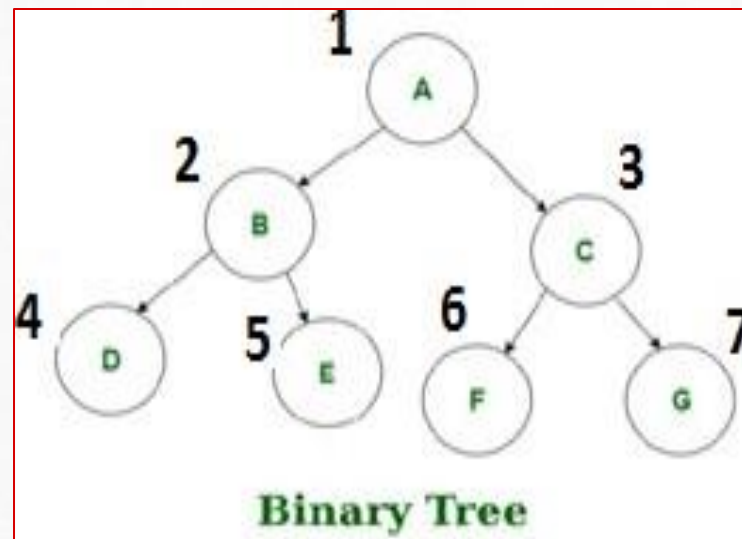
Binary Tree Representations

A binary tree data structure is represented using two methods. Those methods are as follows...

- a) Array Representation**
- b) Linked List Representation**

Representation of Binary Trees using Arrays

- ✓ The sequential representation uses an array for the storage of tree elements (nodes).
- ✓ The number of nodes a binary tree has defines the size of the array being used.
- ✓ The ROOT node of the binary tree lies at the array's first index.
- ✓ The index at which a particular node is stored will define the indices at which the right and left children of the node will be stored. An empty tree has NULL or ZERO as its first index



Height of the given Binary Tree, $h = 2$

Total number of nodes of given binary tree = $2^{h+1} - 1$

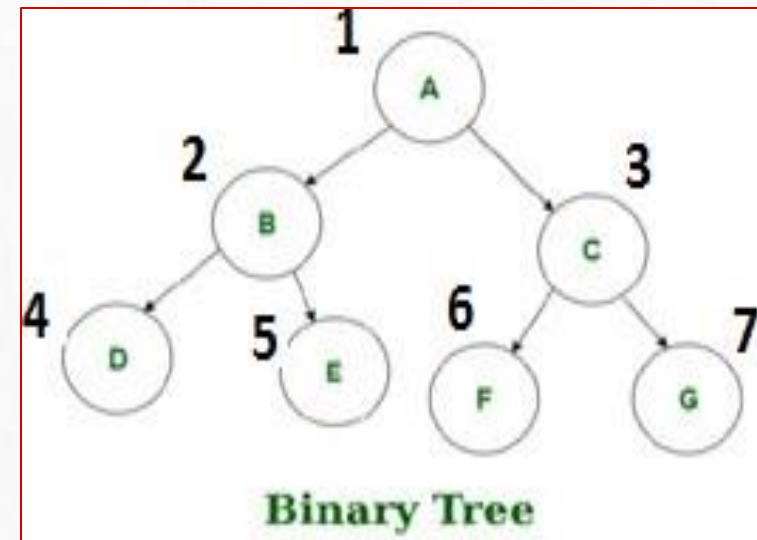
$$= 2^{2+1} - 1$$

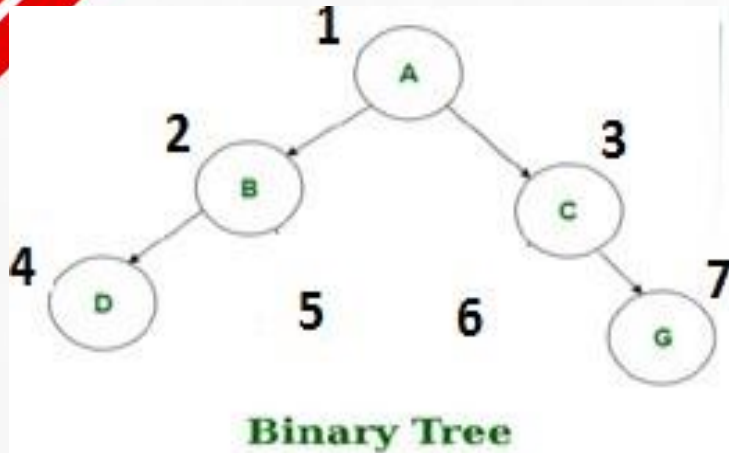
$$= 2^3 - 1$$

$$= 8 - 1 = 7$$

Therefore, the size of Array = **7** (i.e., Maximum number of nodes)

1	2	3	4	5	6	7
A	B	C	D	E	F	G





Height of the given Binary Tree, $h = 2$

Total number of nodes of given binary tree = $2^{h+1} - 1$

$$= 2^{2+1} - 1$$

$$= 2^3 - 1$$

$$= 8 - 1 = 7$$

Therefore, the size of Array = 7 (i.e., Maximum number of nodes)

1	2	3	4	5	6	7
A	B	C	D	---	----	G
				-	-	

Advantages:

- **Static data storage:** Direct access to any node is possible due its static memory allocation.
- **Random Access:** To find the parent, left child or right child of any particular node is fast because of random access.

Disadvantages:

- **Memory Wastage:** wastage of memory due to its memory allocation for missing elements.
- **Implementation:** It depends on height of the tree and as well as total nodes of the tree.
- **Insertion and Deletion operation:** The insertion and deletion of any node in the tree will be costlier (difficult) as other nodes have to be adjusted at appropriate positions

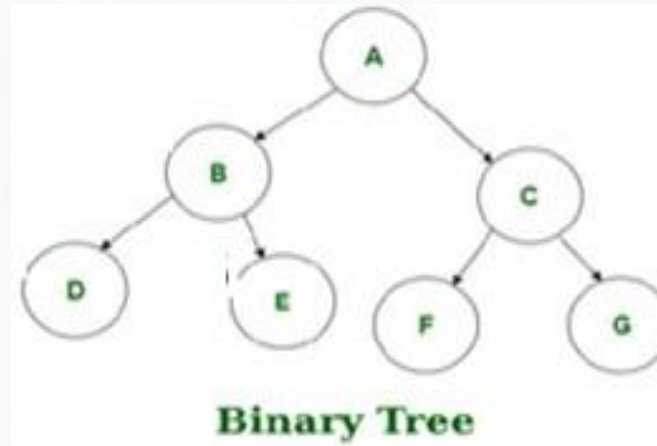
Representation of Binary Trees using Linked List

- We use a double linked list to represent a binary tree.
- In a double linked list, every node consists of **three fields**;
 - ✓ First field for storing left child address,
 - ✓ Second for storing actual data and
 - ✓ Third for storing right child address

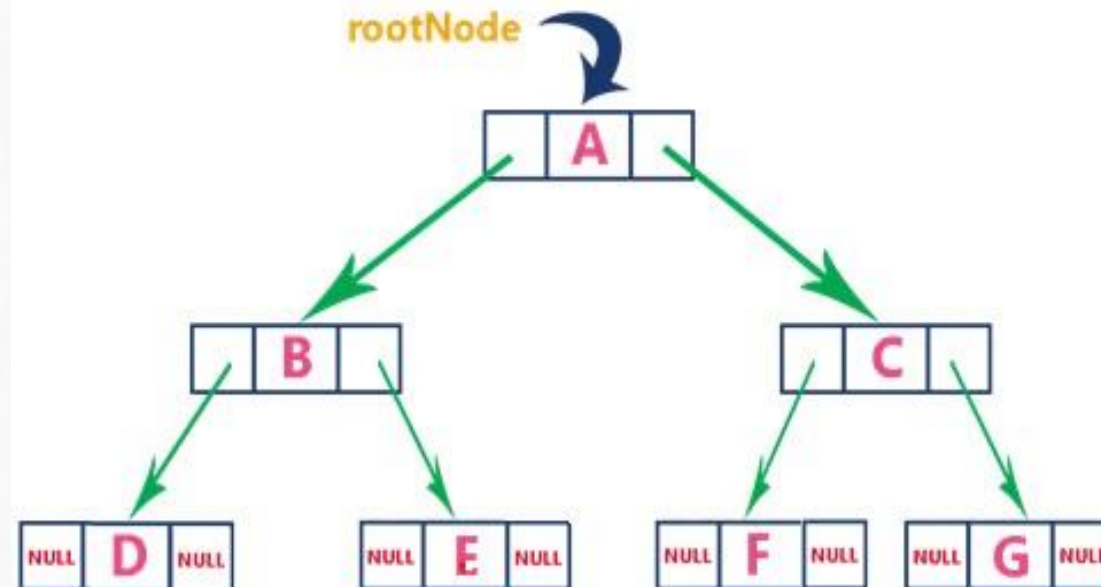
In this linked list representation, a node has the following structure:



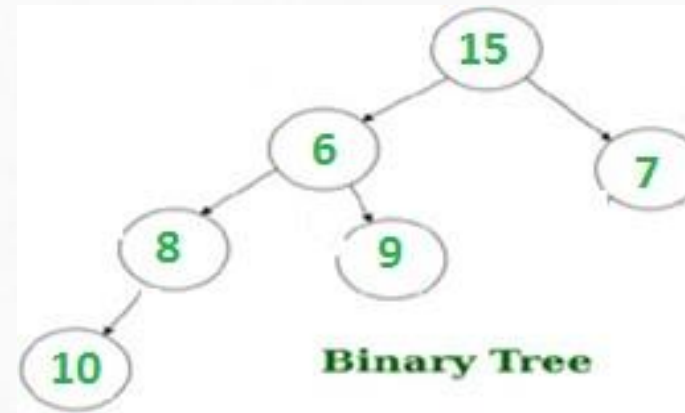
Example:1



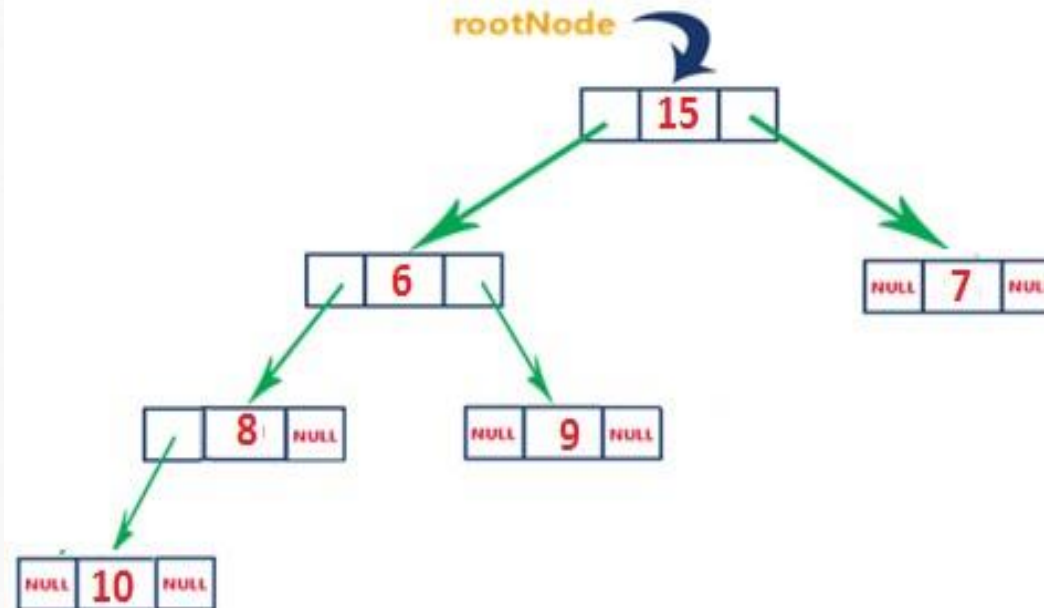
The above example of the binary tree represented using Linked list representation is shown as follows...



Example-2:



The above example of the binary tree represented using Linked list representation is shown as follows



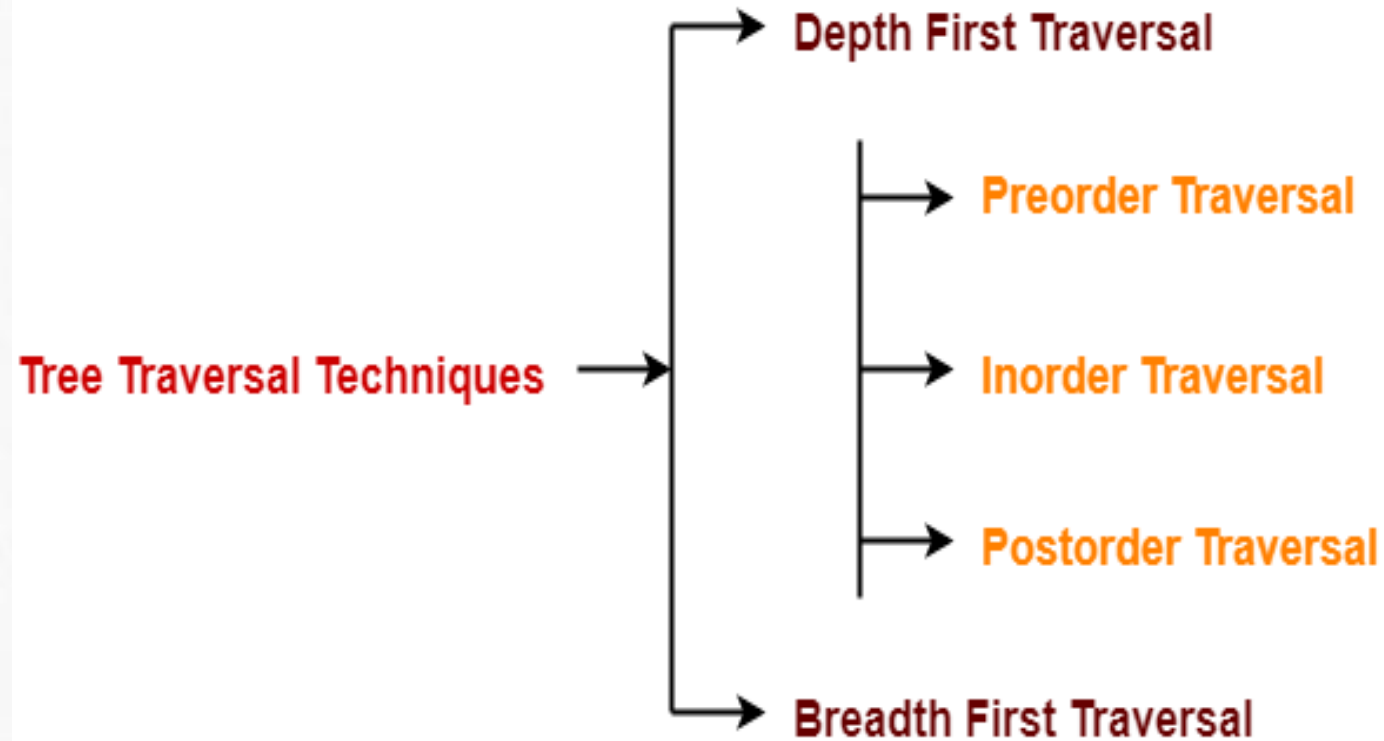
Advantages:

- **Dynamic data storage:** it can grow and shrink at runtime by allocating and deallocating memory.
- **No Memory Wastage:** Efficient memory utilization can be achieved since the size of the linked list increase or decrease at runtime.
- **Insertion and Deletion operation:** Insertion and deletion operations are quite easier in the linked list. There is no need to shift elements after the insertion or deletion of an element only the address present in the next pointer needs to be update.

Disadvantages:

- **Memory Usage:** Extra memory is required in the **linked list** as compared to an array. Because in a linked list, **a pointer is also required** to store the address of the next element and it require extra memory for itself.
- **Traversal:** Direct or random accessing to an element is not possible in a linked list due to its dynamic memory allocation. That means extra memory is used for pointers with every element

Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal



In-order Traversal: (left Child - root - right Child)

Algorithm:

Until all nodes are traversed (visited)

Step-1: Recursively traverse Left sub-tree // `inorder(root->left)`

Step-2: Visit ROOT node // `display(root->data)`

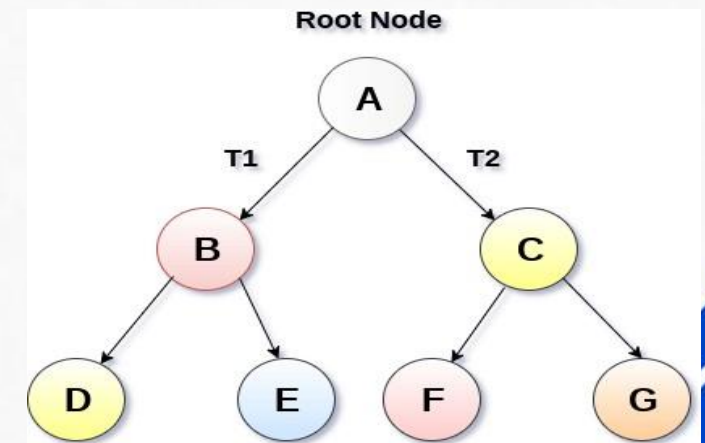
Step-3: Recursively traverse Right sub-tree // `inorder(root->right)`

Example-1:

- We start from A, and following in-order traversal.
- We move to its left sub-tree first; that is B.
- B is also traversed in-order.
- The process goes on until all nodes are visited.

The In-order traversal of the above binary tree is

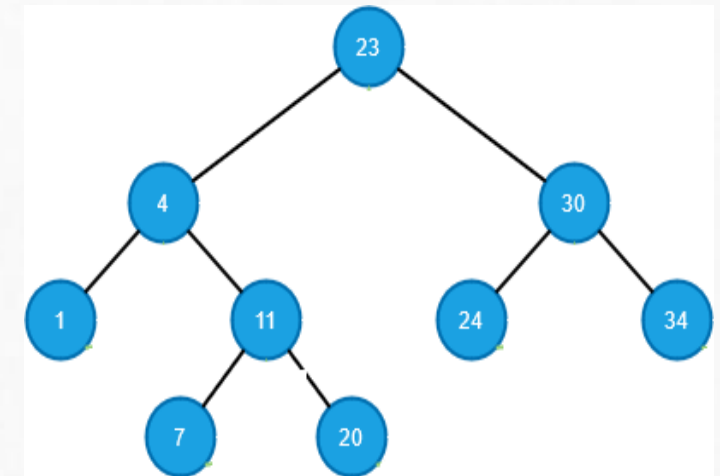
D → B → E → A → F → C → G



Example-2:

- We start from 23, and following in-order traversal.
- We move to its left sub-tree first; that is 4.
- **4 is also traversed in-order.**
- The process goes on until all nodes are visited.
- The In-order traversal of the above binary tree is

$1 \rightarrow 4 \rightarrow 7 \rightarrow 11 \rightarrow 20 \rightarrow 23 \rightarrow 24 \rightarrow 30 \rightarrow 34$



2. Pre - Order Traversal (root – left Child – right Child):

Algorithm:

Until all nodes are traversed (visited)

Step-1: Visit ROOT node // `display(root->data)`

Step-2: Recursively traverse Left sub-tree // `inorder(root->left)`

Step-3: Recursively traverse Right sub-tree // `inorder(root->right)`

Example-1:

→ We start from A, and following pre-order traversal.

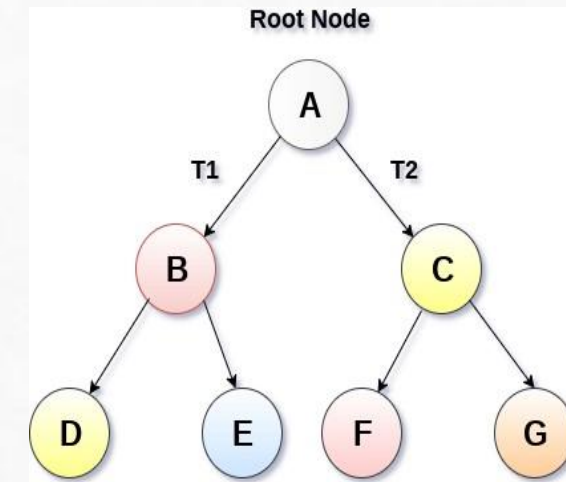
→ We move to its left sub-tree first; that is B.

→ **B is also traversed pre-order.**

→ The process goes on until all nodes are visited.

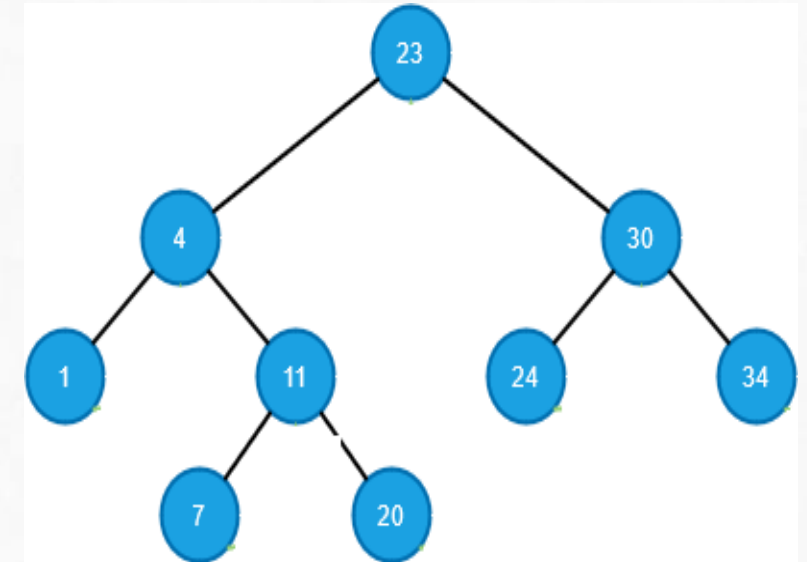
The pre-order traversal of the above binary tree is

A → B → D → E → C → F → G



Example-2:

- We start from 23, and following pre-order traversal
 - We move to its left sub-tree first; that is 4.
 - 4 is also traversed pre-order.
 - The process goes on until all nodes are visited
- The pre-order traversal of the above binary tree is
23 → 4 → 1 → 11 → 7 → 20 → 30 → 24 → 34



Post - Order Traversal (left Child – right Child - root)

Algorithm:

Until all nodes are traversed (visited)

Step-1: Recursively traverse Left sub-tree // `inorder(root->left)`

Step-2: Recursively traverse Right sub-tree // `inorder(root->right)`

Step-3: Visit ROOT node // `display(root->data)`

Example-1:

→ We start from A, and following post-order traversal.

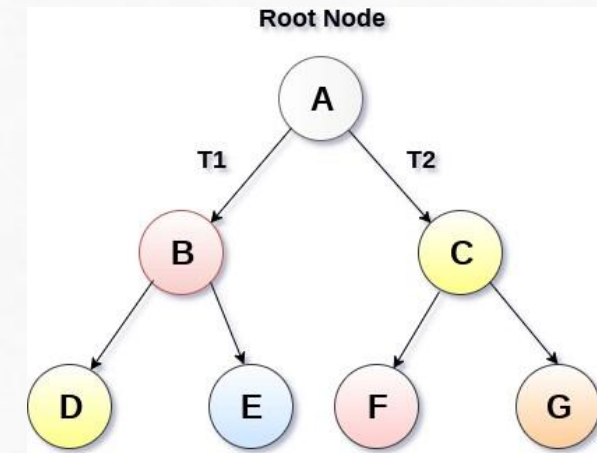
→ We move to its left sub-tree first; that is B.

→ **B is also traversed post-order.**

→ The process goes on until all nodes are visited.

The post-order traversal of the above binary tree is

D → E → B → F → G → C → A

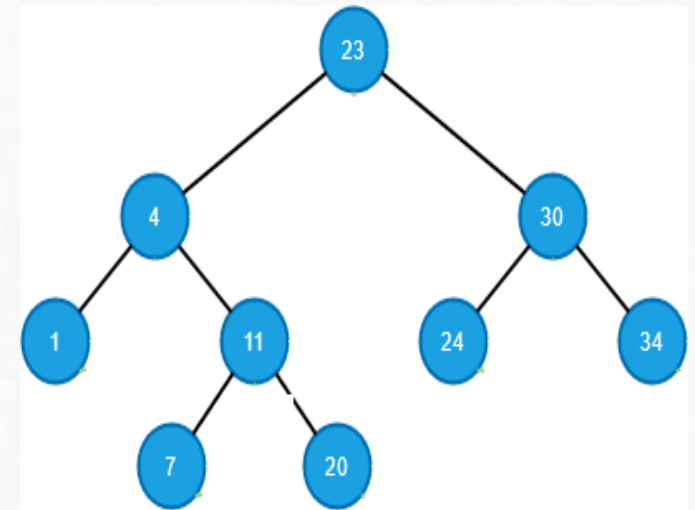


Example-2:

- We start from 23, and following post-order traversal.
- We move to its left sub-tree first; that is 4.
- **4 is also traversed post-order.**
- The process goes on until all nodes are visited.

The post-order traversal of the above binary tree is

1 → 7 → 20 → 11 → 4 → 24 → 34 → 30 → 23



Construct a Binary Tree using **In-order** and **Post-order** Tree traversals

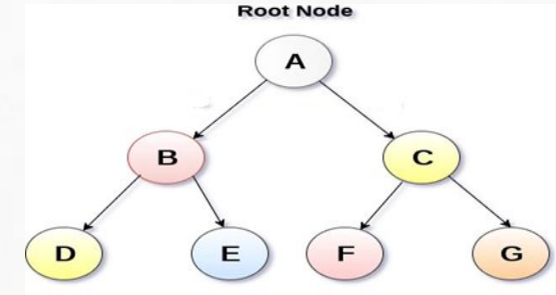
Algorithm:

1. Find the root node using the post-order traversal.
2. Find the left sub trees and the right sub trees using in-order traversal by finding the index of the root node of respective sub trees.
3. Once the root node is found, we can recurse down on the left and right sub trees, i.e., left sub array, and right sub array split at respective root index to repeat the same process until we find at most a single element in either sub-array.

Example-1: To Construct a binary tree using the following In-order and Post-order traversal of the binary tree:

In-order: $D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$

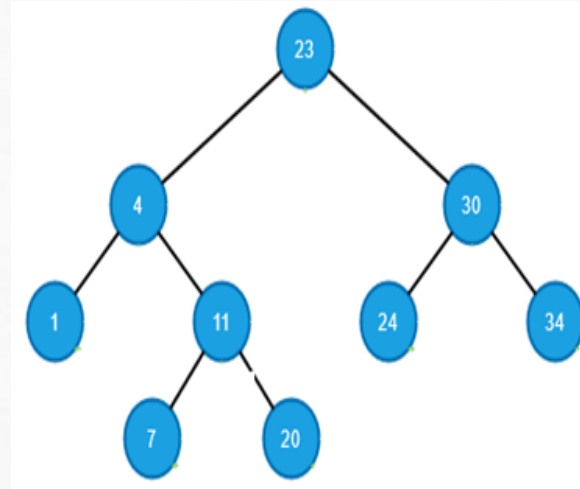
Post-order: $D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$



Example-2: To Construct a binary tree using the following In-order and Post-order traversals of the binary tree:

In-order: $1 \rightarrow 4 \rightarrow 7 \rightarrow 11 \rightarrow 20 \rightarrow 23 \rightarrow 24 \rightarrow 30 \rightarrow 34$

Post-order: $1 \rightarrow 7 \rightarrow 20 \rightarrow 11 \rightarrow 4 \rightarrow 24 \rightarrow 34 \rightarrow 30 \rightarrow 23$



Construct a Binary Tree using **In-order and Pre-order** Tree traversals

Algorithm:

1. Find the root node using the pre-order traversal.
2. Find the left sub trees and the right sub trees using in-order traversal by finding the index of the root node of respective sub trees.
3. Once the root node is found, we can recurse down on the left and right sub trees, i.e., left sub array, and right sub array split at respective root index to repeat the same process until we find at most a single element in either sub-array

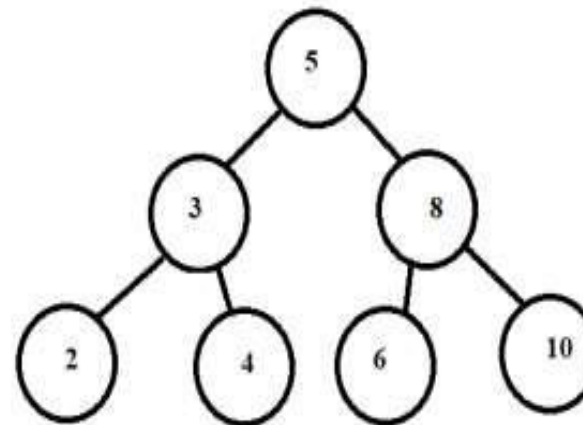
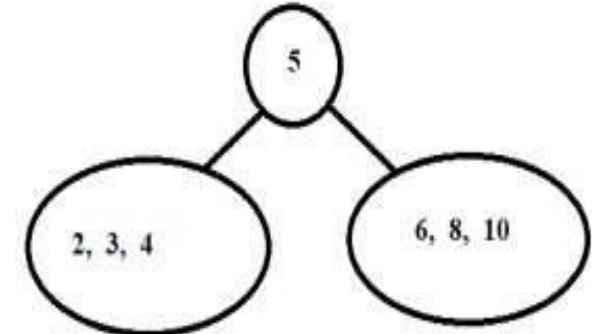
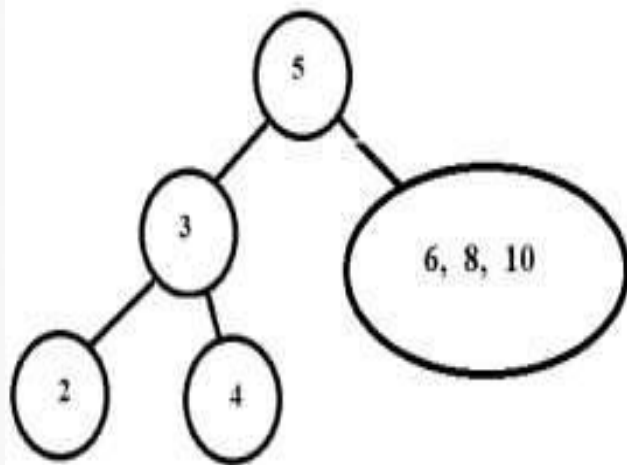
Example-1: Construct binary tree for given preorder and in order traversals.

In order: 2-3-4-5-6-8-10

Preorder: 5-3-2-4-8-6-10

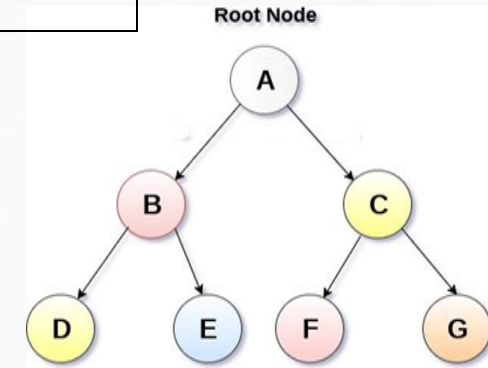
In order

2-3-4 $\leftarrow$ 5 $\rightarrow$ 6-8-10



Example:2: Construct binary tree for given post order and in order traversals.

In-order: $D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$
Pre-order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$



Example:3 Construct binary tree for given post order and in order traversals.

In-order: $1 \rightarrow 4 \rightarrow 7 \rightarrow 11 \rightarrow 20 \rightarrow 23 \rightarrow 24 \rightarrow 30 \rightarrow 34$
Pre-order: $23 \rightarrow 4 \rightarrow 1 \rightarrow 11 \rightarrow 7 \rightarrow 20 \rightarrow 30 \rightarrow 24 \rightarrow 34$

